

```
var world = new b2World(
    new b2Vec2(0, 10) //gravity
    , true
);
```

Once this is done, we'll add a ground surface. We create objects and define their state using `b2BodyDef` and then create fixtures using `b2Fixture` and define their physical properties like density, friction, restitution etc. using `b2FixtureDef`. An entire list of all parameters that can be used if available in the documentation.

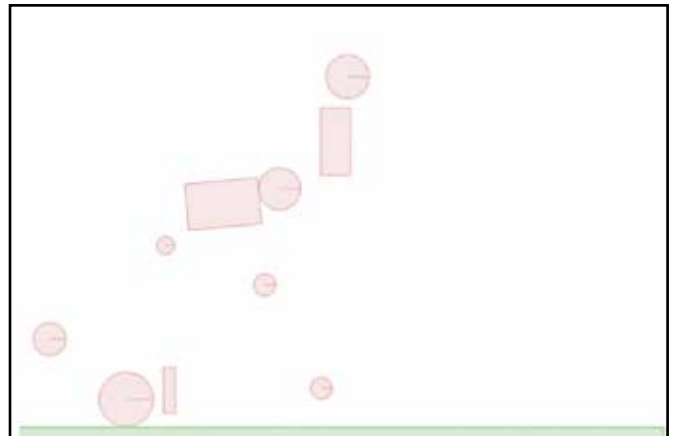
```
var fixDef = new b2FixtureDef;
fixDef.density = 1.0;
fixDef.friction = 0.5;
fixDef.restitution = 0.2;
```

```
var bodyDef = new b2BodyDef;
//create ground
bodyDef.type = b2Body.b2_staticBody;
bodyDef.position.x = 9;
bodyDef.position.y = 13;
fixDef.shape = new b2PolygonShape;
fixDef.shape.SetAsBox(10, 0.5);
world.CreateBody(bodyDef).CreateFixture(fixDef);
```

You can use the same definition to create walls and enclosures. Now we'll add some objects that have random characteristics. We're going to be adding ten objects and we'll be randomising their shape (quadrilaterals or circles). Further on, even the length/width/radius will be randomised along with the co-ordinates where they are to be initialised.

```
bodyDef.type = b2Body.b2_dynamicBody;
for(var i = 0; i < 10; ++i) { //creating 10 objects
    if(Math.random() > 0.5) { //randomising shape using a condition
        fixDef.shape = new b2PolygonShape;
        fixDef.shape.SetAsBox(
            Math.random() + 0.1 //set width of box
            , Math.random() + 0.1 //set height of box
        );
    } else {
        fixDef.shape = new b2CircleShape(
            Math.random() + 0.1 //set radius of circle
        );
    }
    bodyDef.position.x = Math.random()
    * 10; //initial position x-co-ordinate
    bodyDef.position.y = Math.random()
    * 10; //initial position y-co-ordinate
    world.CreateBody(bodyDef).CreateFixture(fixDef);
}
```

An optional step is to setup debugdraw, which paints all the objects so you can observe interactions and subsequently debug. You can also mark out object that are active and non-active using different colours.



Our randomly generated canvas

```
var debugDraw = new b2DebugDraw();
debugDraw.SetSprite(document.
    getElementById("canvas").getContext("2d"));
debugDraw.SetDrawScale(30.0);
debugDraw.SetFillAlpha(0.3);
debugDraw.SetLineThickness(1.0);
debugDraw.
    SetFlags(b2DebugDraw.e_shapeBit | b2DebugDraw.e_jointBit);
world.SetDebugDraw(debugDraw);

window.setInterval(update, 1000 / 60);
```

The last thing that needs to be done is to update the world whenever actions are performed. These can be user generated actions or actions resulting from incidental collisions of objects in the game world.

```
function update() {
    world.Step(
        1 / 60 //frame-rate
        , 10 //velocity iterations
        , 10 //position iterations
    );
    world.DrawDebugData();
    world.ClearForces();
};
```

To summarise, we've created a world, implemented gravity along a vector. A ground surface was added as a rigid body and then ten random objects were generated that would drop down and collide with the ground. This is the simplest implementation of the JavaScript library and we'd love to see any works that you've created using the same library. >

Resources:

Project Home

<https://code.google.com/p/box2dweb/>

Box2D API Documentation

<http://www.learn-cocos2d.com/api-ref/1.0/Box2D/html/index.html>

Example featured in the article

<http://pastebin.com/Qa94fJRW>